



Assesment Empleabilidad - Cohorte 6

Propósito

El propósito de esta prueba técnica es evaluar la capacidad del coder para construir una solución fullstack profesional basada en una plataforma interna de mensajería, integrando base de datos relacional, backend, frontend, autenticación, seguridad a nivel de datos e inteligencia artificial con recuperación aumentada por contexto. La prueba busca evidenciar análisis de negocio, normalización hasta 3FN, lógica crítica dentro de PostgreSQL, arquitectura limpia, experiencia de usuario responsiva y un copiloto de IA que responda únicamente con información permitida para el usuario autenticado.

Metodología:

Tiempo y duración: La prueba técnica tiene una duración de 8 horas, en jornada observada, con 3 descansos de 20 minutos que no cuentan dentro del tiempo. El coder debe priorizar los requisitos del mvp, justificar recortes en DECISIONS.md y demostrar criterio técnico durante la ejecución.

Rúbricas de evaluación

- Consulta en Moodle la rúbrica oficial de la prueba para validar los criterios de evaluación y ponderación.

Reglas

- Prohibido el plagio.
- La prueba tiene una duración máxima de 8 horas continuas, con los descansos definidos por la jornada observada.
- Puedes usar documentación oficial, ejemplos de código abierto y herramientas de IA como apoyo, evitando el plagio.
- Debes tener la capacidad de explicar tu código durante la sustentación.
- Debes realizar la prueba de forma individual.

Requerimientos para la sustentación técnica

- Hacer un documento que explique todos los requerimientos técnicos descritos en la prueba.



Descripción del Proyecto

Riwi Co. S.A.S. requiere modernizar su comunicación interna mediante una plataforma de mensajería organizada, segura y consistente. El sistema debe administrar usuarios, mensajes, estados de lectura, búsqueda de conversaciones y consultas a un copiloto de IA. Adicionalmente, los mensajes deben de poderse eliminar o editar conservando sus estados originales en caso de fallo. El requisito no negociable es que ningún usuario pueda leer, buscar o consultar mediante el copiloto contenido a los que no tiene acceso.

Requerimientos Técnicos

1. Análisis, normalización y modelo de datos

- Construye un Modelo Entidad Relación con entidades, atributos, claves primarias, claves foráneas, cardinalidades y justificación del tipo de clave elegido.
- Crear un corpus seed.json que identifique entidades, relaciones y reglas de negocio implícitas, y documente el proceso de normalización hasta Primera, Segunda y Tercera Forma Normal.

2. Implementación de base de datos en PostgreSQL

- Implementa una base de datos PostgreSQL 15 o superior con nombre `bd_nombre_apellido_clan`.
- Todos los nombres de tablas y columnas deben estar en inglés e iniciar con el prefijo `rw_`.
- Incluye DDL completo, PK, FK con ON DELETE explícito y justificado, UNIQUE, al menos un índice único parcial, NOT NULL, CHECK y fechas timestampz en UTC.

3. Lógica de negocio en la base de datos

- Implementa funciones transaccionales, garantizando que los permisos se validen en la base de datos y que no existan rastros parciales ante errores.
- Activa Row Level Security sobre canales y mensajes, usando un rol de aplicación sin BYPASSRLS y un actor fijado por transacción mediante `app.current_user_id`.
- Crea la vista de conversaciones del usuario.
- Crea mínimo dos procedimientos almacenados: consulta de usuarios y un procedimiento de para la edición y eliminación de usuarios.

4. Búsqueda, recuperación de contexto y seguridad

- Delimitar la forma en la cual el copiloto debe de recuperar los mensajes de cada usuario. No debe de tener acceso a los mensajes globales, únicamente los canales dónde el actor es miembro.
- Usar una base vectorial para guardar los mensajes y un motor de embeddings para recuperarlos con el LLM.
- Incorporar al menos un trigger para mantener el vector de búsqueda consistente.
- Se prohíbe el borrado físico de mensajes, el SQL por concatenación y la paginación con OFFSET.





5. Backend y API REST

- Aplica Clean Architecture con capas explícitas y dependencias apuntando al dominio; el dominio no debe depender del framework web ni del driver de base de datos.
- Implementa casos de uso delgados que validen entrada, invoquen funciones de base de datos y mapeen resultados.
- Incluye principios SOLID demostrables en el propio código, y evalúa si es necesario aplicar un patrón de diseño. En caso de aplicarlo, justificar correctamente.
- Expón una API REST con códigos de estado correctos, manejo uniforme de errores, identificador de correlación y paginación por keyset.

6. Autenticación y autorización

- Implementa inicio de sesión con los usuarios, verificando contraseñas contra hash seguro.
- Usa JWT con token de acceso de vida corta y refresh token con rotación, almacenado de forma segura.
- Protege las rutas y toma el identificador del usuario exclusivamente del token, nunca del cuerpo de la petición.
- Propaga el actor autenticado a las funciones de base de datos y a las políticas RLS.

7. Frontend

- Construye una interfaz con mínimo tres zonas: conversación, panel del copiloto y perfil de usuario.
- Permite envío de mensajes con estados pendiente, enviado y fallido.
- Implementa carga de historial de forma diferida preservando la posición del scroll, estados de carga, vacío y error.
- La interfaz debe ser responsiva en móvil y escritorio, estar disponible en español e inglés y evitar cadenas incrustadas en componentes.

8. Copiloto de IA

- Integra un copiloto de IA con enfoque RAG, recuperando contexto exclusivamente perteneciente al actor que lo esté usando.
- Cada respuesta debe incluir citas a los mensajes fuente y responder con honestidad cuando no exista contexto suficiente.
- El copiloto debe conocer al usuario autenticado, su nombre y cargo, construyendo ese contexto en el servidor desde el token.
- El proveedor de IA debe ser intercambiable entre diferentes proveedores eligiendo una interfaz específica como OpenAI SDK.
- El system prompt debe estar versionado, el contenido de chats debe tratarse como dato no confiable y deben existir negativas explícitas por falta de permisos, fuera de alcance o contexto insuficiente.



9. QA, evidencias y extras

- Incluye mínimo dos pruebas automatizadas contra PostgreSQL real: una para verificar que se rechaza a un usuario no miembro y otra para confirmar que no se retorna mensajes de canales privados ajenos.
- Entrega evidencias en capturas o video de máximo 5 minutos mostrando inicio de sesión, envío de mensaje, búsqueda, respuesta del copiloto con citas y negativa correcta ante una pregunta sin permisos, todo enfocado a un pitch comercial.

10. Despliegue

- docker compose up debe levantar base de datos, backend y frontend.
- Debe existir un comando documentado para ejecutar migraciones y cargar el corpus completo.
- Incluye .env.example sin secretos reales y verifica que el proyecto pueda levantarse en una máquina limpia siguiendo únicamente el README.

11. Consultas y funciones SQL requeridas

- Consulta 1: historial de mensajes de un canal con paginación por keyset.
- Consulta 2: búsqueda de mensajes con resaltado del término encontrado.
- Consulta 3: recuperación de contexto para el copiloto con permisos en SQL.
- Consulta 4: consumo acumulado del copiloto por usuario.





Entregables

- Script DDL, scripts de carga, scripts DML, consultas SQL, funciones, triggers, vistas, procedimientos y políticas RLS.
- Modelo Entidad Relación en PDF o imagen, seed.json original y archivos utilizados para representar la solución.
- Documentación de API mediante Swagger/OpenAPI publicado o colección Postman exportada.
- README.md, ARCHITECTURE.md, DECISIONS.md, evidencias de ejecución y URL del repositorio.

Criterios de aceptación y condiciones de invalidación

Criterios de aceptación

- El modelo representa correctamente el negocio y llega hasta 3FN.
- La lógica crítica vive en PostgreSQL mediante transacciones, restricciones, RLS, funciones, vistas y procedimientos.
- La carga del corpus funciona correctamente.
- La API, autenticación JWT, frontend responsivo, internacionalización y copiloto funcionan de punta a punta.
- La mensajería debe funcionar en tiempo real.

Condiciones que invalidan la prueba

- Las contraseñas quedan almacenadas en texto plano.



- El primer commit contiene lógica previa al inicio de la jornada.
- El coder no puede explicar el código entregado o el repositorio deriva de una aplicación de mensajería existente.

